

Natural Language Processing

Session 07 – BERT

Prof. Dr. Richard Sieg
SoSe 2026



Schedule

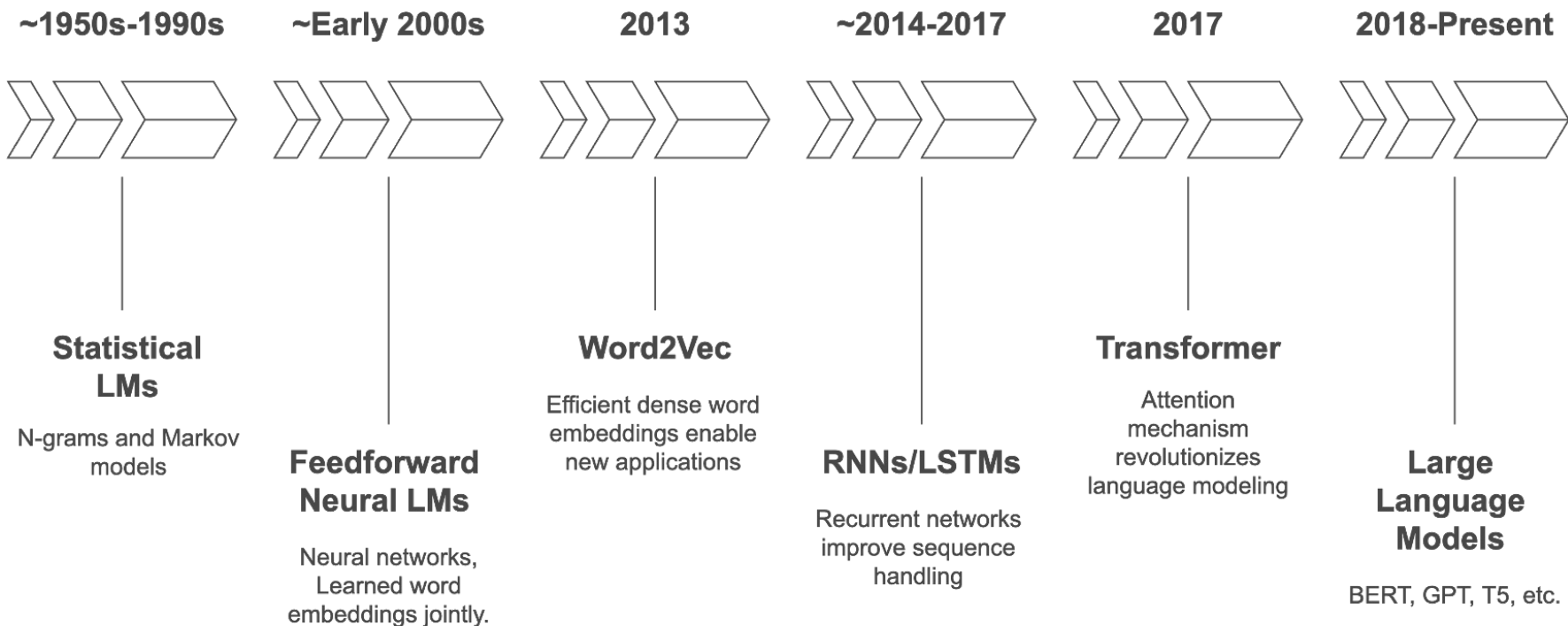
Date	Weekday	CW	Room	Topic
16.04.	Thursday	16	149	Introduction: The World of NLP
23.04.	Thursday	17	149	Text Processing Fundamentals
30.04.	Thursday	18	149	Linguistic Annotations & Pattern Matching
05.05.	Tuesday	19	154	Text Vectorization: From Text to Numbers ⚠️ Substitute for 14.05.
07.05.	Thursday	19	149	Text Classification
28.05.	Thursday	22	149	Language Models, Word Vectors + 🎤 Guest Lecture Ines Montani
01.06.	Monday	23	390	📝 Exams — Master DS Students
02.06.	Tuesday	23	154	BERT: Pre-Training & Fine-Tuning ⚠️ Substitute for 04.06.
11.06.	Thursday	24	149	The BERT Ecosystem
18.06.	Thursday	25	149	Introduction to LLMs
25.06.	Thursday	26	149	Working with LLMs
02.07.	Thursday	27	149	Ethics, Limitations & Exam Preparation
09.07.	Thursday	28	149	📝 Exams (afternoon) — Bachelor DIS Students
10.07.	Friday	28	149	📝 Exams (afternoon) — Bachelor DIS Students

Today:
Feedback Sessions with my
Coach Imke Buß

Your feedback is very valuable and
anonymized. Please be very honest.

Thank you 🙌

History of Language Models



Attention Is All You Need

Ashish Vaswani*

Google Brain

avaswani@google.com

Noam Shazeer*

Google Brain

noam@google.com

Niki Parmar*

Google Research

nikip@google.com

Jakob Uszkoreit*

Google Research

usz@google.com

Llion Jones*

Google Research

llion@google.com

Aidan N. Gomez* †

University of Toronto

aidan@cs.toronto.edu

Łukasz Kaiser*

Google Brain

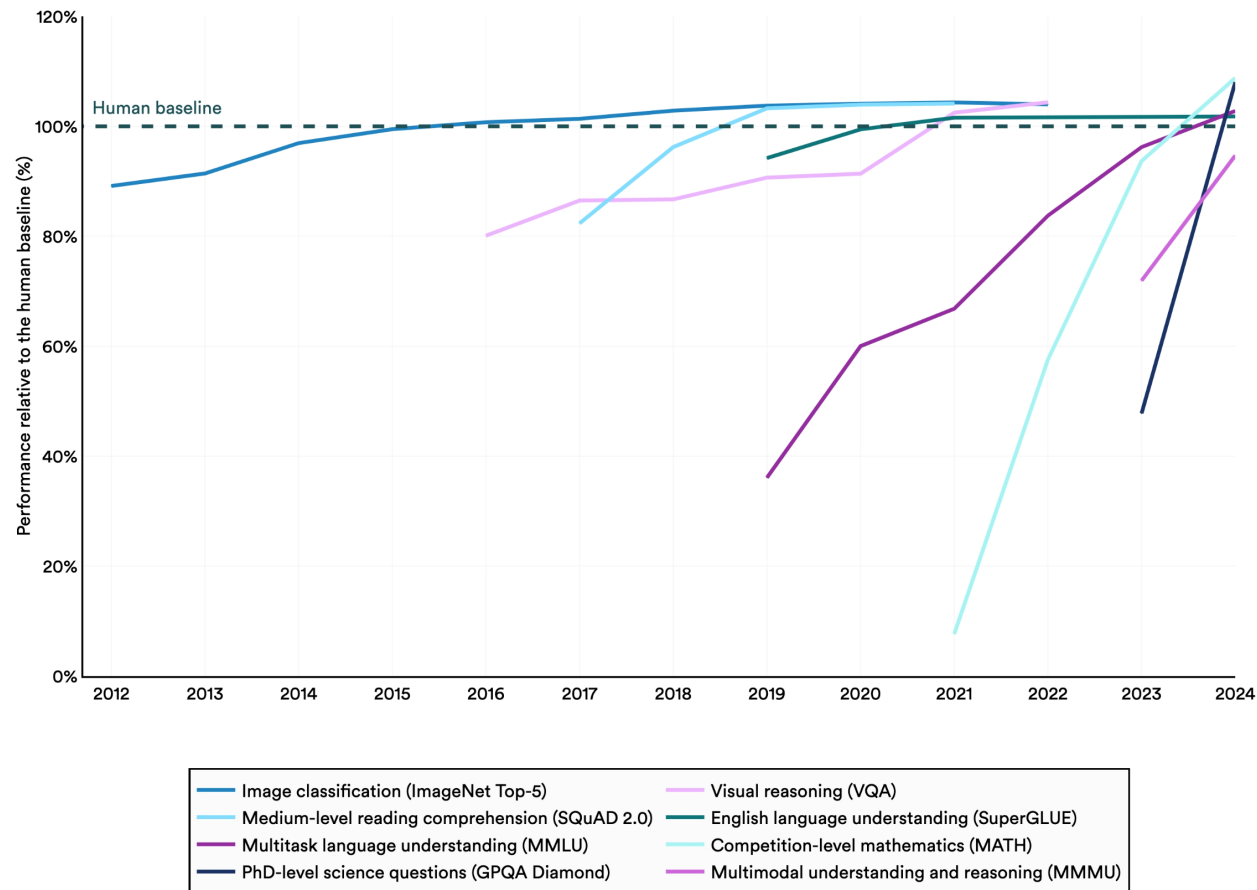
lukaszkaiser@google.com

Illia Polosukhin* ‡

illia.polosukhin@gmail.com

Select AI Index technical performance benchmarks vs. human performance

Source: AI Index, 2025 | Chart: 2025 AI Index report





Inside an
LLM

Agenda

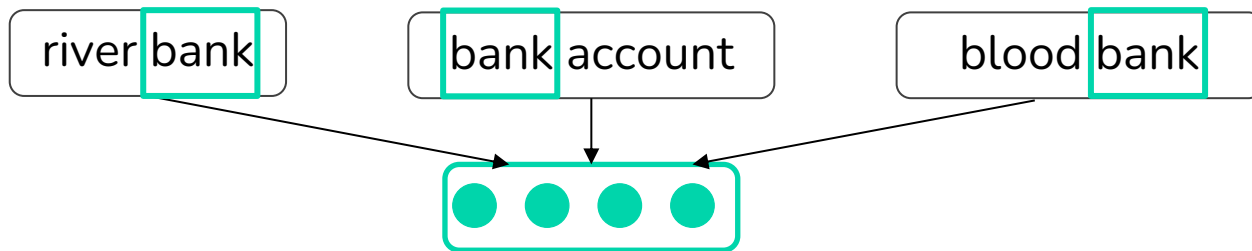
01. Contextualized Embeddings
02. Attention & The Transformer
03. Pre-Training
04. BERT
05. Fine-Tuning BERT

01.

Contextualized
Embeddings

Recap: Static Word Vectors

- Last session: Word2Vec & GloVe: dense vectors that capture word meaning.
 - One vector per word type (one dictionary entry $\rightarrow$ one vector)
 - Trained on raw text, no labels needed
 - Geometry encodes meaning: similar words $\rightarrow$ nearby vectors
- The vector for a word is fixed, it is the same no matter where or how the word is used.
- We then feed these fixed vectors into a task-specific model (LogReg, NN...) and train that model from scratch for each task.



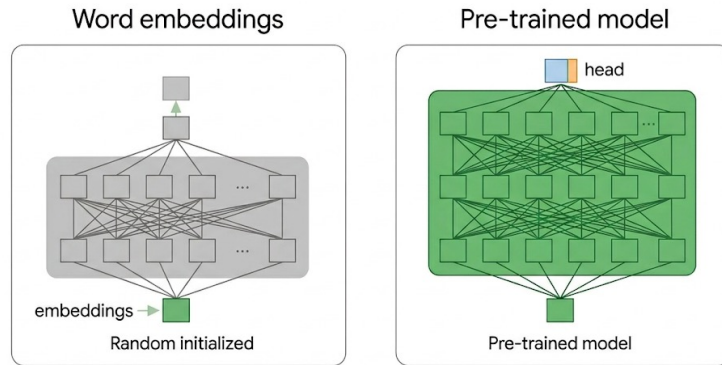
Problem 1: Meaning Depends on Context

- Words are ambiguous. The same form carries different meanings.
 - **mouse** → a small rodent / a device to control a cursor
 - **bank** → a financial institution / the side of a river
- A single static vector must average all senses into one point. It cannot commit to the sense actually intended.
- Even grammar needs context:
 - *"The chicken didn't cross the road because it was too tired."* → it = the chicken
 - *"The chicken didn't cross the road because it was too wide."* → it = the road
- The word it is identical; only the context tells us the referent.

Problem 2: We Waste the Model

- *We throw the model away:*
 - Word2Vec/GloVe give us only an embedding layer. The network that learned them is discarded. Every new task still needs its own architecture, trained largely from scratch.
- *Pre-training is barely leveraged:*
 - Knowledge transfer is limited to that one embedding layer. The rest of the task model starts with random parameters. It has no initial idea of how language works (syntax, long-range structure, world knowledge).

GOAL: Transfer knowledge through the whole model, and let each word's representation depend on its context.



Two Great Ideas

- The jump from word vectors to modern models is really two ideas
- **Idea 1:** From words to words-in-context
 - Instead of representing a word in isolation, represent it together with its context.
 - Word2Vec / GloVe → CoVe / ELMo
- **Idea 2:** From replacing embeddings to replacing whole models
 - Instead of only swapping the embedding layer, pre-train an entire model and reuse all of it.
 - CoVe / ELMo → GPT / BERT

This is often called the "ImageNet moment" of NLP: a pre-trained network as a powerful, ready-made initialization.

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* [†]
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* [‡]
illia.polosukhin@gmail.com

02.

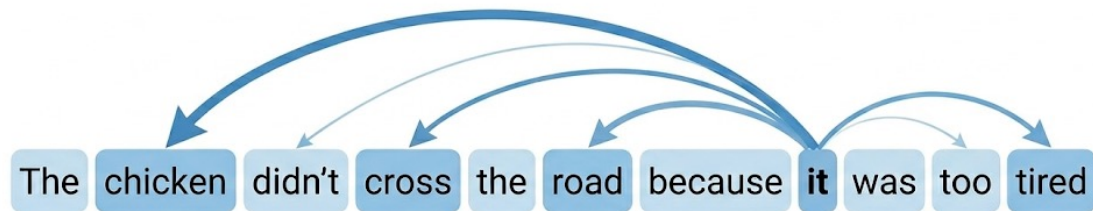
Attention & The Transformer

The Idea of (Self-)Attention

- We want each word's representation to depend on the other words in the sentence
- Attention lets each word look at every other word in the sentence and focus on the ones relevant to it, then update its own representation.

For a given word, attention works in three steps:

1. Compare it to every other word and compute a relevance score for each
2. Turn the scores into weights with a softmax (weights are positive and sum to 1)
3. Build its new representation as the weighted sum of the other words' representations



Three Roles: Query, Key, Value

When we compute the new representation of one word (the current word), every word in the sentence takes on three roles:

Role	What it represents
Query (q)	what the current word is looking for
Key (k)	how relevant each word (including the word itself) is to that query
Value (v)	what each word actually contributes to the new representation

- Attention then works as a lookup: the query is compared against every key to decide how much each word matters, and those amounts are used to mix the values into the new representation.
- Each input vector x_i is turned into its query, key, and value vector by learned weight matrices:

$W_q, q_i = W_q x_i$ Queries $W_k, k_i = W_k x_i$ Keys. $W_v, v_i = W_v x_i$ Values

Computing Attention

1. **Score** every query against every key with a dot product (high dot product means similar):

$$\text{score}(x_i, x_j) = \frac{q_i \cdot k_j}{\sqrt{d_k}}$$

The $\sqrt{d_k}$ scaling keeps dot products from getting too large in high dimensions.

2. **Normalize** the scores into weights with a softmax:

$$\alpha_{ij} = \sigma(\text{score}(x_i, x_j))$$

3. **Output**: Sum the values, weighted by α :

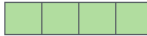
$$z_i = \sum_j \alpha_{ij} v_j$$

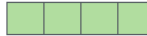
Input

Thinking

Machines

Embedding

x_1 

x_2 

Queries

q_1 

q_2 

Keys

k_1 

k_2 

Values

v_1 

v_2 

Score

$q_1 \cdot k_1 = 112$

$q_1 \cdot k_2 = 96$

Divide by 8 ($\sqrt{d_k}$)

14

12

Softmax

0.88

0.12

Softmax

X

Value

v_1 

v_2 

Sum

z_1 

z_2 

Multi-Head Attention

- One attention computation is one “head”. Transformers use many heads in parallel.
- Each head has its own matrices W^Q, W^K, W^V
- *Intuition*: each head can focus on a different kind of relationship, such as syntax, coreference, or position
- Heads run independently, then their outputs are concatenated and projected back to the original dimension.
- Splitting attention into heads keeps the total model size the same. It just lets the model attend to several things at once.
- Original Transformer: input model dim $d = 512$, 8 heads, each of dim 64.

1) This is our input sentence*

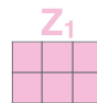
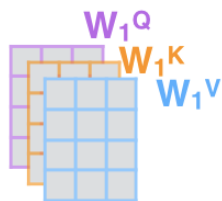
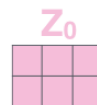
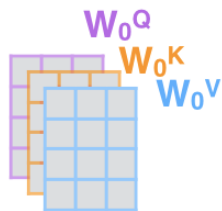
2) We embed each word*

3) Split into 8 heads.
We multiply X or R with weight matrices

4) Calculate attention using the resulting $Q/K/V$ matrices

5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

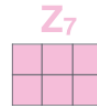
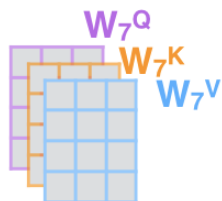
Thinking
Machines



...

...

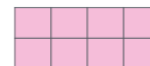
...



W^O



Z



* In all encoders other than #0,
we don't need embedding.
We start directly with the output
of the encoder right below this one



Masked vs. Bidirectional Attention

- Attention, by default, lets a word see all other words, both to its left and its right.
- But when the task is to predict the next word, seeing the right side is cheating
- To fix this we use **masking**: before the softmax, set the scores for all future words to $-\infty$.
- Softmax turns $-\infty$ into 0, so those words contribute nothing.
- **Masked attention**: each word sees only the words to its left. Needed for predicting the next word.
- **Bidirectional attention**: each word sees the whole sentence, left and right. Better when the goal is to understand a sentence rather than continue it.

	<Start>	is	all	you	need
<START>		$-\infty$	$-\infty$	$-\infty$	$-\infty$
is			$-\infty$	$-\infty$	$-\infty$
all				$-\infty$	$-\infty$
you					$-\infty$
need					

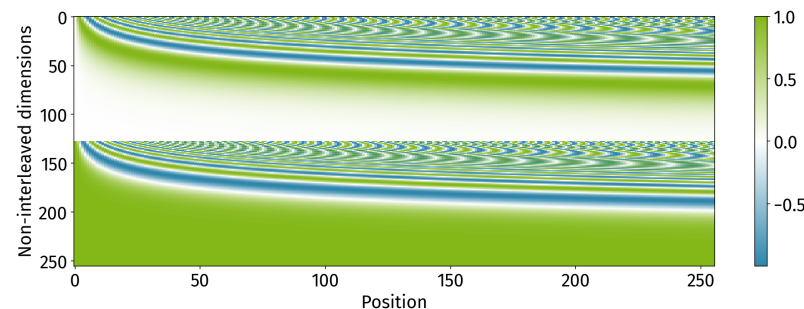
<START>					
is					
all					
you					
need					

Where Order Comes From: Positional Embeddings

- Attention is a weighted sum, so it has no inherent sense of order.
- *Fix*: explicitly add position information to each token.

$$\tilde{x}_i = \underbrace{x_i}_{\text{token embedding}} + \underbrace{p_i}_{\text{positional embedding}}$$

- Two common ways to get p_i :
- Learned positional embeddings: one trainable vector per position (used by BERT)
- Fixed sinusoidal embeddings: sine/cosine of varying frequencies; no parameters, extrapolates to longer sequences (original Transformer)

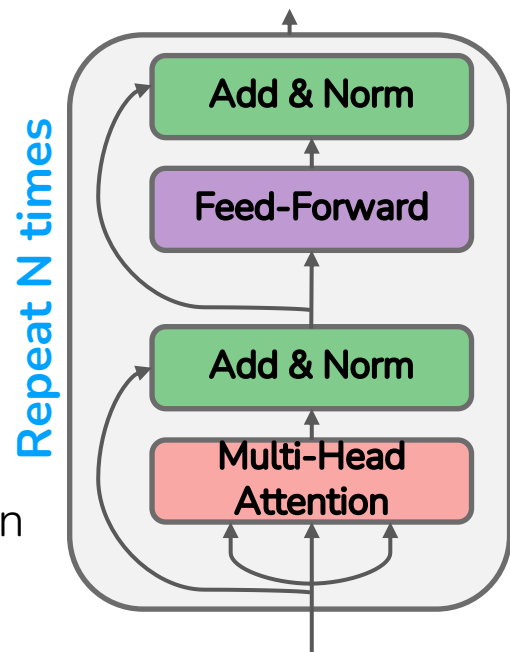


The Transformer Block

The Transformer is built entirely from attention, with no other sequence-processing machinery. Because all words are processed in parallel rather than one after another, it trains very efficiently on large datasets.

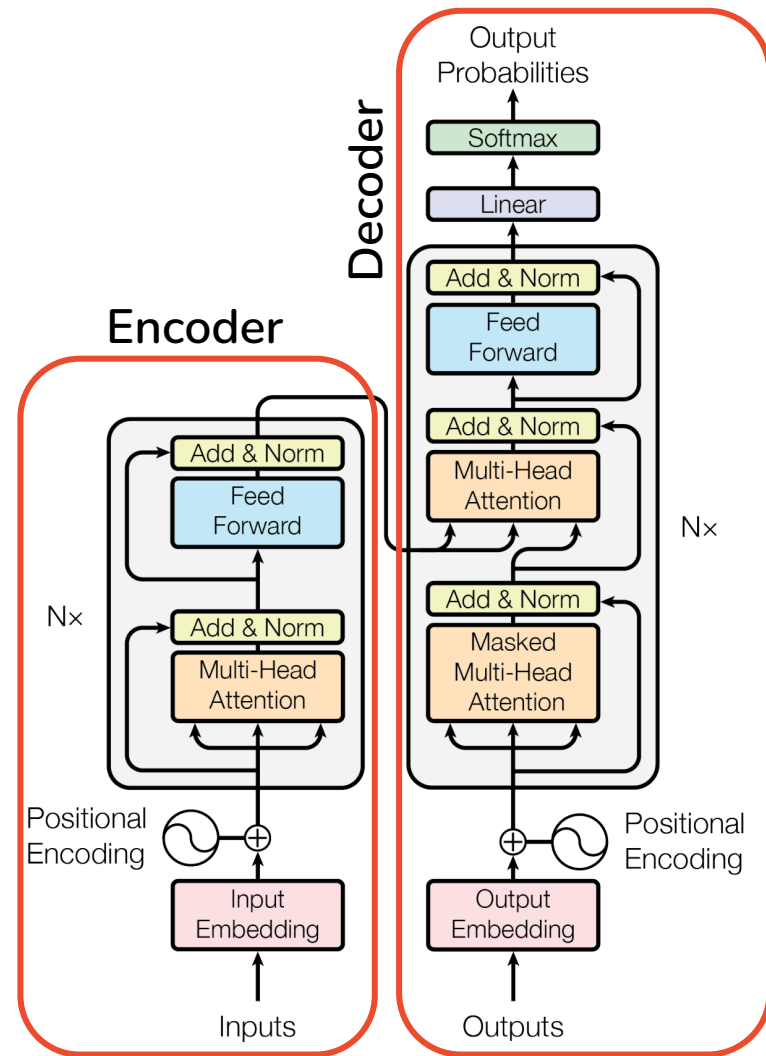
A transformer block stacks four ingredients:

1. Multi-head attention: the only part that mixes information across tokens
2. Feed-forward network: a 2-layer MLP applied to each position independently, to "process what attention gathered"
3. Residual connections: add a sublayer's input to its output, for easier gradient flow and deeper stacks
4. Layer normalization: per-token normalization, for stable training



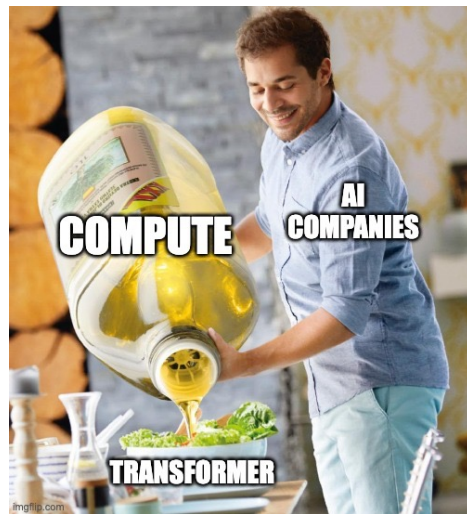
The Full Transformer

- Input $\rightarrow$ token embeddings + positional embeddings
- N transformer blocks $\rightarrow$ contextualized representations, layer by layer
- Output head $\rightarrow$ a final layer that maps representations to predictions
- **Decoder:** A transformer trained with masked attention learns to predict the next word, so it can generate text. This is the GPT family.
- **Encoder:** A transformer trained with bidirectional attention sees the whole sentence at once, so it is built to understand text. This is the BERT family.

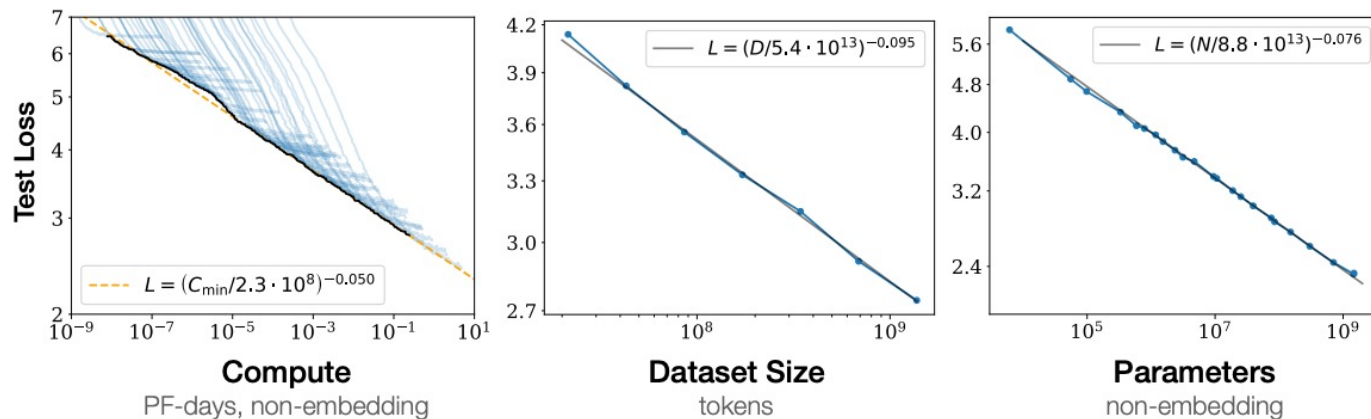


The Success of the Transformer

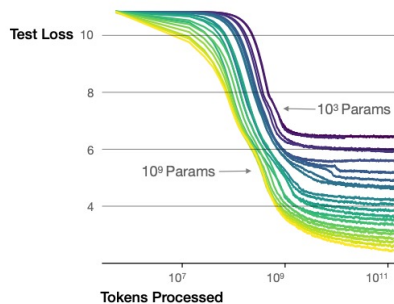
- For Machine Translation, the Transformer beat all former State of the Art models while using much less training time.
- As it turns out, the Transformer architecture is amazing for pre-training models on huge datasets
- And the Encoder (BERT etc) or Decoder (GPT etc) themselves make great Language Models
- The rest is scaling (although attention has quadratic costs)



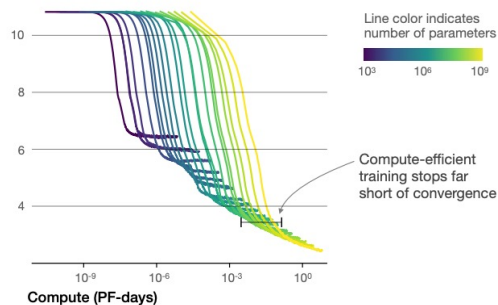
Scaling Transformer Models



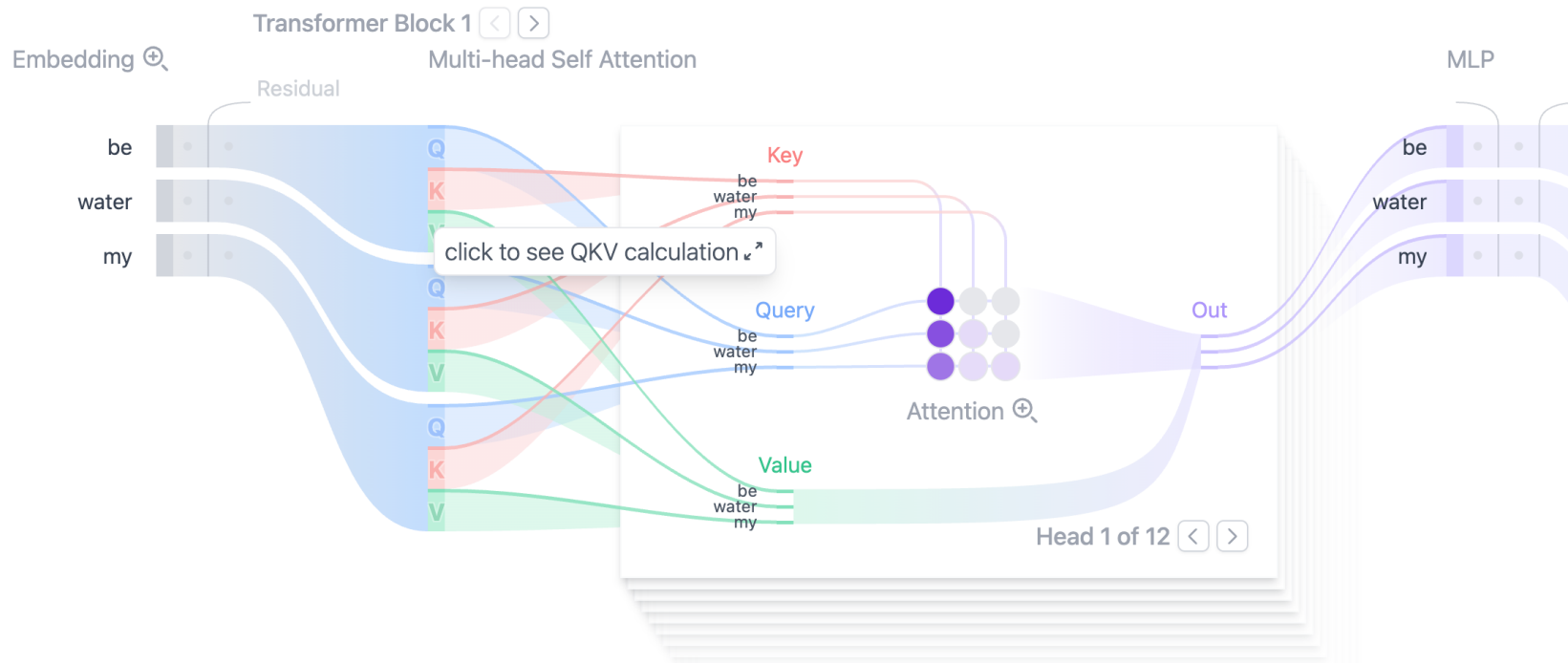
Larger models require **fewer samples** to reach the same performance



The optimal model size grows smoothly with the loss target and compute budget



Folks do great visualizations of the transformer



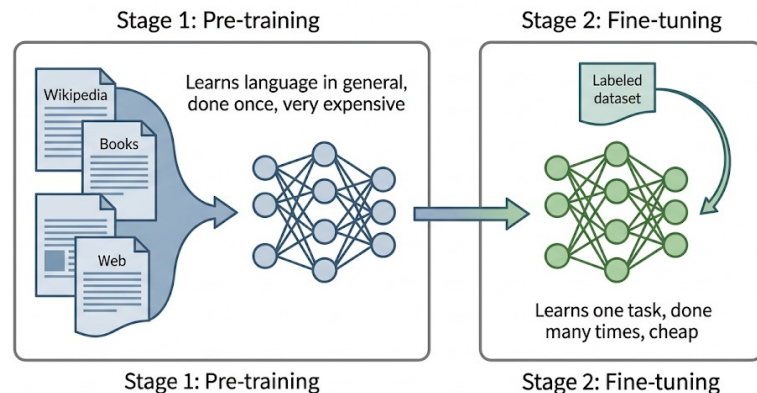
Transformer Explainer

03.

Pre-Training

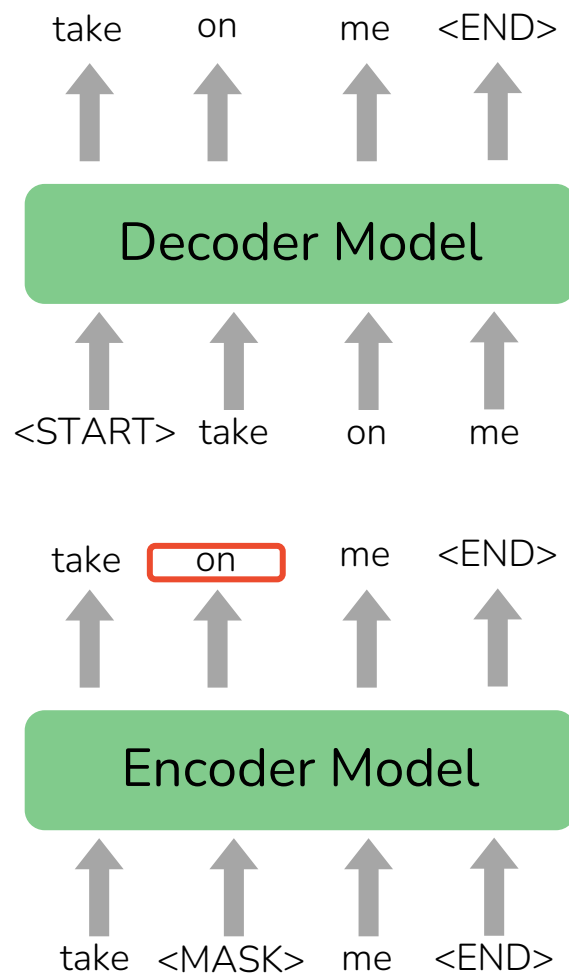
The Problem with Training from Scratch

- So far, every task we have built starts almost from zero.
- Word vectors transfer, but the rest of the model has random parameters
- The model has no idea how language works before training begins: no grammar, no word senses, no world knowledge
- It must learn all of that plus the actual task, from a small labeled dataset
- Labeled data is expensive.
Raw text is everywhere.
- **The pre-training idea:** first let a model learn language in general from huge amounts of raw text. Then teach it your specific task.



Self-Supervision: Learning Without Labels

- Pre-Training a model works by masking some parts of the input and train the model to reconstruct these parts
- This allows us to process massive amounts of data without any labels (unsupervised)
- Two pre-training styles
- **Masked:** Pretraining through language modeling, i.e. masking the next word and let the model predict the next word
→ Decoder Models (GPT etc)
- **Bidirectional:** Predict missing words in the middle like a cloze
→ Encoder Models (BERT etc)



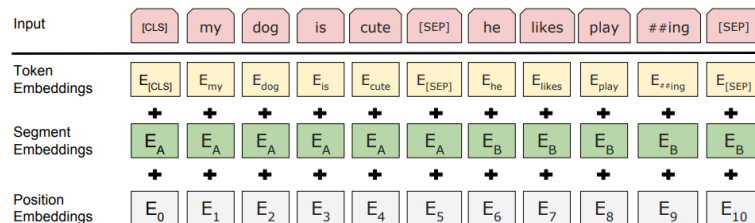
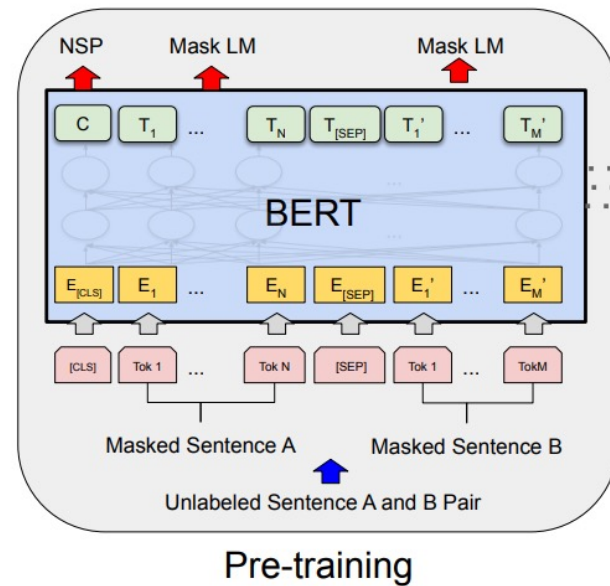
04.

BERT



BERT: Bidirectional Encoder Representations from Transformers

- A stack of transformer blocks using bidirectional attention: every word sees the whole sentence (Encoder style) developed by Google
- Trained on Masked LM and **next sentence prediction**
- Next sentence prediction: predict whether one sentence follows the other or randomly sampled (other BERT variants neglect this)
- Goal is not to generate text but to compute a rich representation of each word in context
- Pre-trained once on huge text, then fine-tuned for understanding tasks

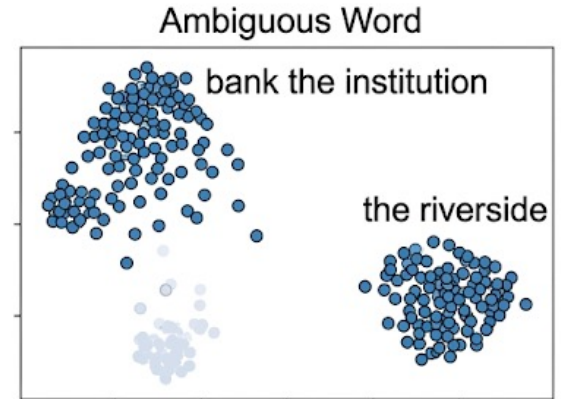
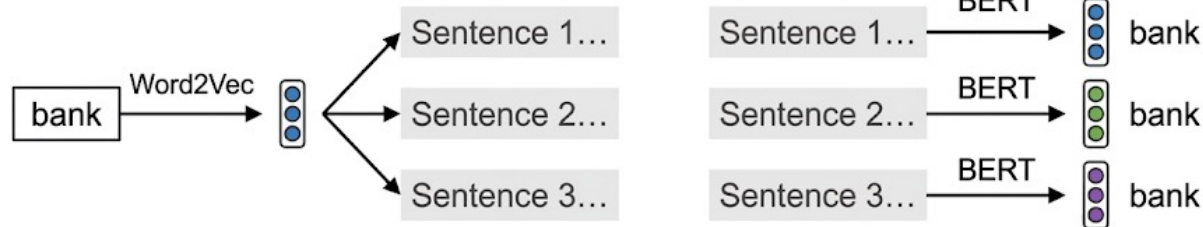


BERT Training Details

- Masked LM details: 15% of tokens are masked and out of these
 - 80% are replaced with [MASK]
 - 10% replaced with random other token
 - 10% are unchanged but still predicted
- BERT-base: 12 layers, 768-dim hidden states, 12 attention heads, 110M params.
- BERT-large: 24 layers, 1024-dim hidden states, 16 attention heads, 340M params.
- Trained on:
 - BooksCorpus (800 million words)
 - English Wikipedia (2,500 million words)
- BERT was pretrained with 64 TPU chips for a total of 4 days.

Where BERT's Word Vectors Come From

- BERT's output vectors are contextual embeddings:
- For each input word, BERT's output vector h_i is that word's meaning in this specific sentence
- The same word in a different sentence gets a different vector: one vector per word instance, not per word type
- These vectors are the result of running the sentence through the encoder, attention has mixed in the surrounding words layer by layer



A Caveat: Raw BERT Vectors and Similarity

- It is tempting to compare two words (or two sentences) by the cosine similarity of their BERT vectors. This works poorly out of the box.
- **Anisotropy:** BERT's vectors nearly all point in similar directions. Two random words can have a cosine close to 1, so cosine no longer separates "related" from "unrelated" well.
- Caused partly by a few "rogue" dimensions with very large values that dominate the cosine and stopwords “dragging” other words into their direction
- Partial fix: standardize the vectors (subtract the mean, divide by the standard deviation)

See SentenceBERT & BERTTopic in the next lecture

05.

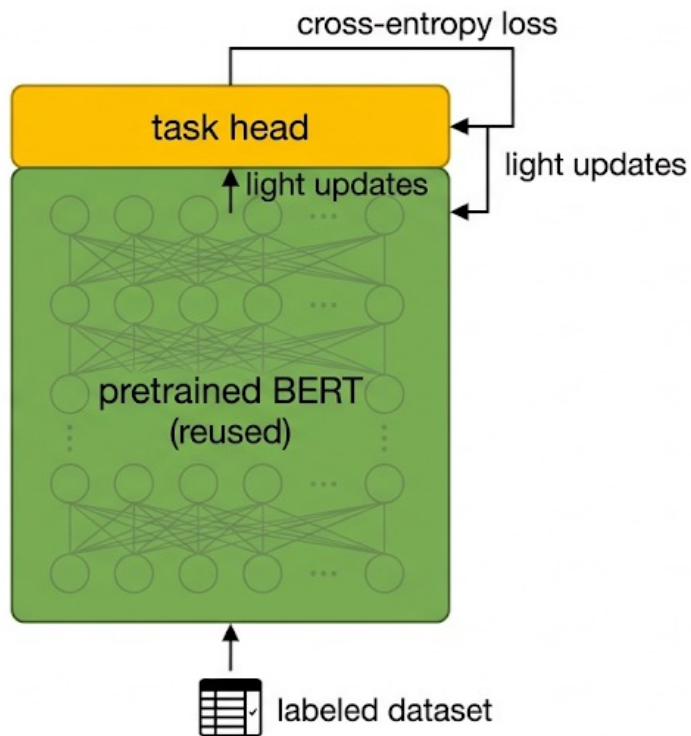
Fine-Tuning BERT



How Fine-Tuning Works

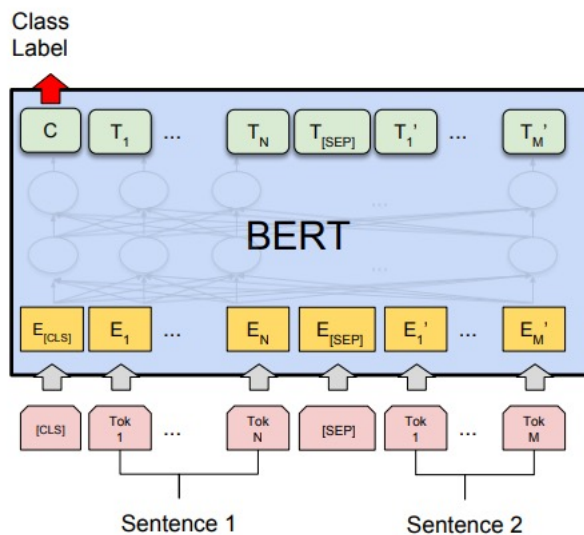
The pretrained BERT already understands language. Fine-tuning just adapts it to a specific task.

1. Take the pretrained BERT (downloaded, not trained from scratch)
 2. Add a small task-specific head on top: usually a single new layer
 3. Train on a labeled dataset for the task
 4. Update the head, and lightly adjust BERT's own weights, using ordinary cross-entropy loss
- This is cheap: small dataset, few passes over the data, often only the top layers change much.

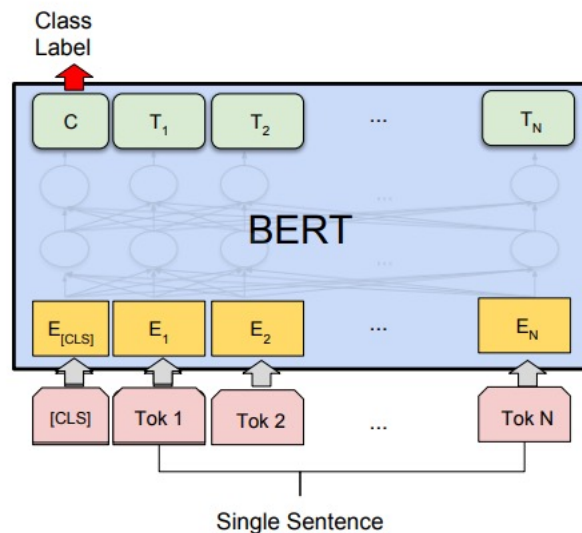


BERT Fine-Tuning Task – Text Classification

- For Text Classification, we start every sentence with a special [CLS] token
- The context vector of this token can then be used in a linear layer and softmaxed

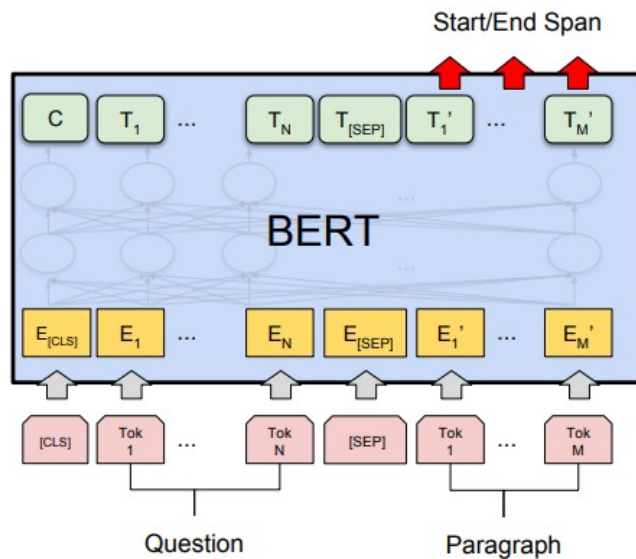


(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG

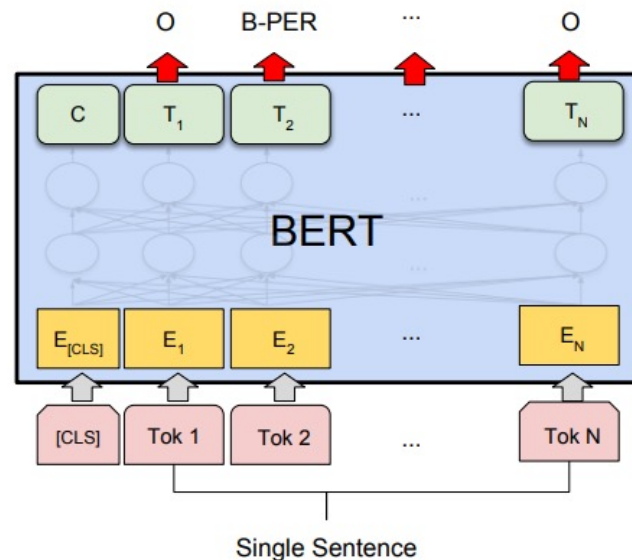


(b) Single Sentence Classification Tasks:
SST-2, CoLA

BERT Fine-Tuning Task – QA & NER Tagging



(c) Question Answering Tasks:
SQuAD v1.1



(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

BERT Success

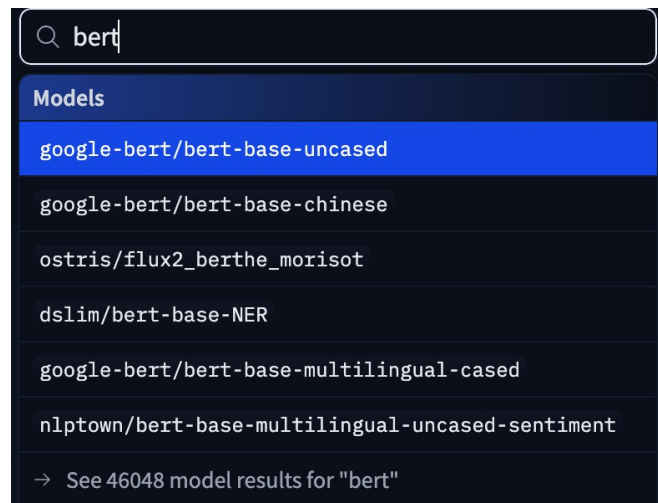
- By fine-tuning, BERT yielded state-of-the-art results on many tasks
- Is considered the powerhouse of modern NLP - in particular its variants
- Use it for any discriminative task in particular text classification

System	MNLI-(m/mm)	QQP	QNLI	SST-2	CoLA	STS-B	MRPC	RTE	Average
	392k	363k	108k	67k	8.5k	5.7k	3.5k	2.5k	-
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
BERT _{BASE}	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	92.7	94.9	60.5	86.5	89.3	70.1	82.1

Table 1: GLUE Test results, scored by the evaluation server (<https://gluebenchmark.com/leaderboard>).

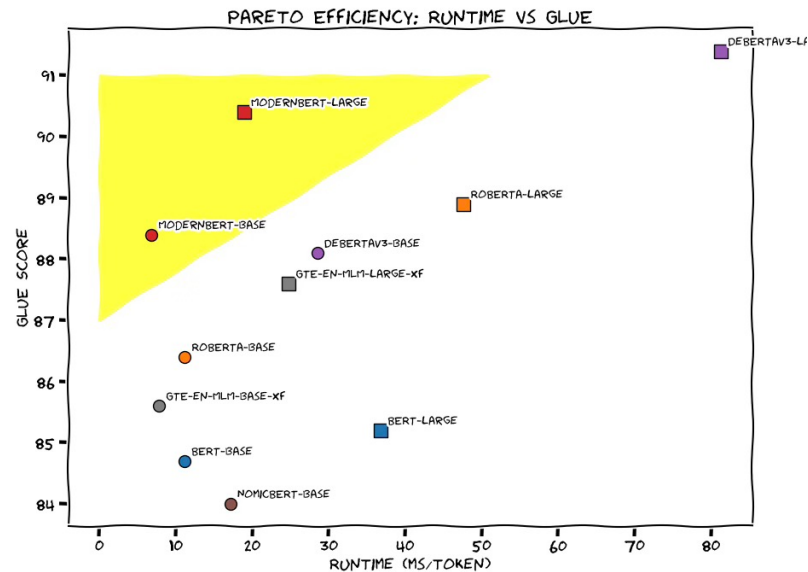
BERT Model Family

- RoBERTa [2019]
 - No next sentence prediction
 - Dynamic masking (other masks every epoch)
 - BPE tokenizer instead of WordPiece
 - Small training adjustments
- DistilBERT
 - Much smaller version (40% less params) trained via student-teacher
- Lots of variants for different languages or use cases, just search HuggingFace 🙌
 - Sentiment analysis, emotion detection
 - German RoBERTa, multilingual variants (XLM-RoBERTa)



modernBERT (12/2024)

- 8192 Tokens in the Context Window (vs 512 Vanilla BERT)
- Improved Transformer architecture using global and local Attention
- Lots of smaller tricks from years of LLM training experience
- EuroBERT (03/2025)
 - Basically, modernBERT trained on European languages



The Hugging Face Ecosystem

- In practice, you rarely build or pre-train these models yourself. Hugging Face is the standard open platform for using them.
- The 🤗 Hub
 - A huge public repository of pretrained models (BERT and many more)
 - Thousands of ready-made datasets
 - Model cards describe each model's training, intended use, and limitations
- The key libraries
 - **transformers**: load any pretrained model and its tokenizer in a few lines
 - **datasets**: download and process datasets efficiently



Access to GPU

For fine-tuning BERT models you need GPU power. You have several options:

1. Our institute has access to an HPC in Mainz and I can give you access to it. Only one slot per team possible. Please send me an email who should get the access.
2. I have access to GPUs on marimo's molab with an RTX Pro 6000 96GB. Contact me.
3. Google Colab in the free version has an okay-ish GPU and for 10\$ per month you can get decent ones.
4. You have a PC with a GPU and/or a MacBook Pro with Silicon Valley Chip.

